```
  def +(other)
    if Roman === other
      other = other.value
    end
    if Integer === other && (other + @value) < MAX_ROMAN
      Roman.new(@value + other)
    else
      x, y = other.coerce(@value)
      x + y
    end
  end

  FACTORS = [["m", 1000], ["cm", 900], ["d", 500], ["cd", 400],
    ["c", 100], ["xc", 90], ["l", 50], ["xl", 40],
    ["x", 10], ["ix", 9], ["v", 5], ["iv", 4],
    ["i", 1]]

  def to_s
    value = @value
    roman = ""
    FACTORS.each do |code, factor|
      count, value = value.divmod(factor)
      roman << (code * count)
    end
    roman
  end
end

iv = Roman.new(4)
xi = Roman.new(11)

iv + 3          # => vii
iv + 3 + 4      # => xi
iv + 3.14159    # => 7.14159
xi + 4900       # => mmmmcmxi
xi + 4990       # => 5001
```

Finally, be careful with `coerce`—try always to coerce into a more general type, or you may end up generating coercion loops. This is a situation where A tries to coerce to B and when B tries to coerce back to A.

Walk the Walk, Talk the Talk

Duck typing can generate controversy. Every now and then a thread flares on social media or someone blogs for or against the concept. Many of the contributors to these discussions have some fairly extreme positions.

Ultimately, though, duck typing isn't a set of rules; it's just a style of programming. Design your programs to balance paranoia and flexibility. If you feel the need to constrain the types of objects that the users of a method pass in, ask yourself why. Try to determine what could go wrong if you were expecting a `String` and instead get an `Array`. Sometimes, the difference is crucially important. Often, though, it isn't. Try erring on the more permissive side for a while, and see whether bad things happen. If not, perhaps duck typing isn't just for the birds.